End-Term Report

Summer of Science 2025

Natural Language Processing (CS 32)

Sandipan Dhali (24B2487)

Mentor: M a n i s h

Contents

1 Introduction to NLP **3**
- 1.1 Language and Human Intelligence . . . 3
- 1.2 Language and Machines . . . 3
- 1.3 Key Applications and Considerations in NLP . . . 3

2 Prerequisites and Foundational Toolkit **4**
- 2.1 Core Python Libraries for Scientific Computing . . . 4
  - 2.1.1 NumPy: The Engine for Numerical Data . . . 5
  - 2.1.2 Pandas: The Toolkit for Data Wrangling . . . 5
  - 2.1.3 Matplotlib: The Framework for Data Visualization . . . 5
- 2.2 Foundations in Machine Learning . . . 6
  - 2.2.1 The Scikit-learn API and Main Workflow . . . 6
  - 2.2.2 A Practical Application: Predicting with a Regressor . . . 7

3 Neural Networks **8**
- 3.1 Introduction to Neural Networks . . . 8
- 3.2 Activation Functions & Backpropagation . . . 9
  - 3.2.1 Activation Functions: . . . 9
  - 3.2.2 Backpropagation: . . . 9
- 3.3 Attention Mechanisms . . . 9
- 3.4 Transformers . . . 9
  - 3.4.1 Architecture: . . . 9
- 3.5 Other Key Topics . . . 10
  - 3.5.1 RNNs & LSTMs: . . . 10
  - 3.5.2 Word Embeddings(More details on NLP Section): . . . 10
  - 3.5.3 Cross-Entropy Loss: . . . 11

4 Word Embeddings and the Word2Vec Algorithm **11**
- 4.1 Word Embeddings . . . 11
- 4.2 Word2Vec: Mathematical Formulation . . . 11
  - 4.2.1 Probability Model: . . . 11
  - 4.2.2 Loss Function: . . . 11
  - 4.2.3 Gradient descent: . . . 11
- 4.3 Negative Sampling . . . 12
- 4.4 Geometric Properties . . . 12
- 4.5 Applications . . . 12

5 Improtant Libraries for Text Preprocessing **12**
- 5.1 NLTK . . . 12
- 5.2 SpaCy . . . 13

6 Text Preprocessing in NLP **13**
- 6.1 Lower Casing . . . 13
- 6.2 Removal of Punctuations . . . 13
- 6.3 Removal of Stopwords . . . 13
- 6.4 Removal of Frequent and Rare Words . . . 14

6.5 Stemming . . . 14
6.6 Lemmatization . . . 14
6.7 Removal of Emojis and Emoticons . . . 14
6.8 Spelling Correction . . . 15

7 Introduction to Tokenization **15**
7.1 Basic Tokenization Features Using NLP (with NLTK) . . . 15
7.2 Tokenization Using TextBlob Library . . . 15
7.3 Some spaCy Tokenization . . . 16
7.4 Gensim Library Tokenization . . . 16

8 Bag of Words Implementation **16**
8.1 Core Concept and Mathematical Foundation . . . 16
8.2 Implementation Steps . . . 17
8.2.1 Text Preprocessing . . . 17
8.2.2 Vocabulary Construction and Frequency Analysis . . . 17
8.2.3 Vector Representation Construction . . . 18
8.3 Advantages and Limitations . . . 19

9 TF-IDF Vectorization **19**
9.1 Mathematical Foundation . . . 19
9.2 Implementation with Scikit-learn . . . 19

10 Basic Sentiment Analysis **20**
10.1 Data Preparation and Feature Engineering . . . 20
10.2 Predicting Sentiment on New Text . . . 21

11 Transformer Architecture **21**
11.1 Core Components . . . 21
11.1.1 Multi-Head Attention Mechanism . . . 21
11.1.2 Positional Encoding . . . 22
11.1.3 Complete Transformer Block . . . 22

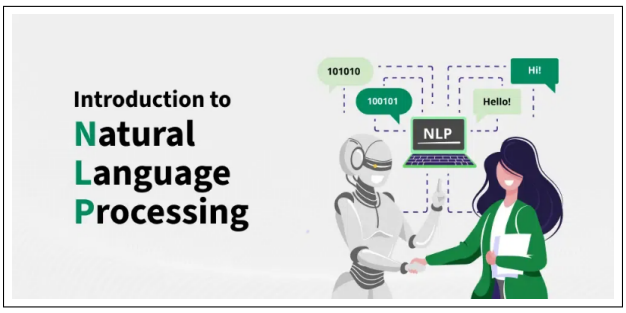
12 Fine-tuning BERT **23**
12.1 Fine-tuning Implementation with Hugging Face . . . 23

13 End-to-End NLP Pipeline **24**
13.1 Pipeline Architecture . . . 24
13.2 Complete Pipeline Implementation . . . 25

14 Model Deployment Basics **25**
14.1 Deployment with FastAPI . . . 25
14.2 Dockerization for Portability . . . 26

1 Introduction to NLP

Natural Language Processing (NLP) is a field of science and engineering focused on developing automatic systems that can understand and generate human language. Its goal is to bridge the gap between human communication and computer understanding.



1.1 Language and Human Intelligence

Human language is a powerful communication tool that enables the efficient sharing and storage of complex ideas, a trait that arguably represents a unique form of intelligence among species.As engineers aiming to build intelligent systems, language is both a fascinating object of study and a critical medium for human-computer interaction, even when other modalities like vision are available.

1.2 Language and Machines

Human children acquire language with remarkable sample and computational efficiency. Despite impressive advances in NLP, modern systems are nowhere near replicating the language acquisition ability of a child. A fundamental and still open problem is the question of *representation*: how should we represent language in a computer so that it can be robustly processed and generated?

This report focuses on the tools provided by deep learning, a highly effective toolkit for representing both the vast diversity of natural language and the underlying rules it often follows. Much of this work will be dedicated to this question of representation, beginning with a foundational sub-question: how do we represent individual words?

1.3 Key Applications and Considerations in NLP

Modern NLP techniques have enabled a wide range of powerful and increasingly ubiquitous applications. However, their failures and limitations can be opaque and difficult to detect. Below are a few major applications:

Machine Translation (MT) One of the earliest successes of NLP, MT systems learn to translate between languages. Research continues to tackle challenges in translating long texts, ensuring contextual correctness, and supporting the vast majority of the world's 7,000 languages.

Question Answering & Information Retrieval This area focuses on building systems that can answer information-seeking questions, from simple facts ("Who is

the emir of Abu Dhabi?") to complex procedural queries. Evolving research aims to broaden the scope of answerable questions and provide verifiable sources for answers.

Summarization and Text Analysis These tools are used to understand public opinion, conduct market research, or simply provide individuals with digestible summaries of complex topics. NLP tools here can be powerful for both increasing access to information and for surveillance, highlighting the "dual use" nature of the technology that practitioners must consider.

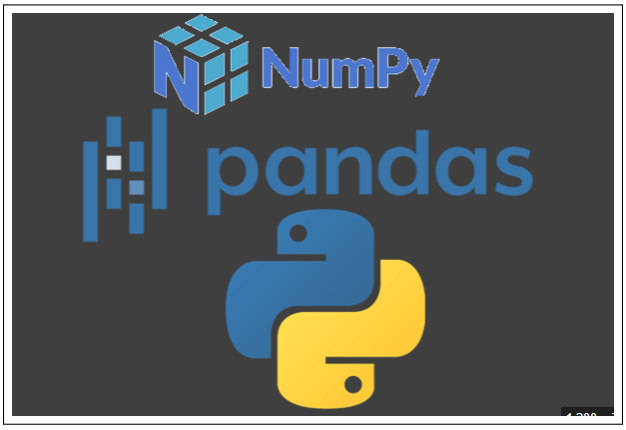
A Note on Scope and Challenges: In all areas of NLP, most existing tools work for a very small number of the world's languages and often fail on marginalized dialects and accents. Furthermore, recent successes have outpaced our ability to audit these systems for biases related to race, gender, and religion, which are often reflected and amplified. With these challenges in mind, we proceed with the goal of building trustworthy systems that can improve people's lives.

2 Prerequisites and Foundational Toolkit

The successful execution of this Natural Language Processing (NLP) project rests upon a robust foundation in computational methods and machine learning theory. This section details the prerequisite knowledge and practical skills acquired in preparation for this work, forming the technical and conceptual bedrock upon which the project is built. The discussion is divided into two core areas: proficiency in Python's scientific computing stack and a formal introduction to machine learning principles, drawing from standard educational materials and documentation.

2.1 Core Python Libraries for Scientific Computing

A significant portion of the preliminary work involved mastering the primary Python libraries that power modern data science and machine learning. These tools provide the necessary infrastructure to ingest, clean, manipulate, and visualize the complex datasets characteristic of NLP tasks.



2.1.1 NumPy: The Engine for Numerical Data

Proficiency in NumPy was developed by treating it as the low-level engine for all numerical tasks. Rather than just theory, the focus was on the practical application of its core object, the ‘ndarray‘.

- **Array Initialization and Manipulation:** Gained experience creating arrays from existing data using `np.array()` and initializing placeholder arrays with `np.zeros()` or `np.ones()`. Mastered inspecting their properties with `.shape` and `.dtype`, and changing their structure for model compatibility using `.reshape()`.
- **Mathematical Operations for ML:** Applied vector and matrix operations that are fundamental to machine learning. This included performing element-wise calculations and, more importantly, using `np.dot()` and the `@` operator for matrix multiplication, which is the core mathematical process inside a neural network layer.
- **Indexing and Slicing:** Learned to efficiently access and modify subsets of data within NumPy arrays. This skill is critical for tasks like splitting a dataset into batches for model training or selecting specific feature columns from a data matrix.

2.1.2 Pandas: The Toolkit for Data Wrangling

Practical skills in Pandas were honed by treating it as the primary tool for the entire data preprocessing workflow, from initial loading to final cleaning.

- **Data Ingestion and Exploration:** Developed a standard workflow for starting any data task: loading a dataset using `pd.read_csv()` and immediately performing an initial inspection using `.head()` to view sample rows, `.info()` to check data types and null counts, and `.describe()` for a statistical summary.
- **Data Cleaning and Text Preprocessing:** This was the most critical application for NLP preparation. Used boolean indexing to filter out irrelevant rows. Handled missing data by identifying it with `.isnull().sum()` and resolving it with methods like `.fillna()` or `.dropna()`. Most importantly, used the `.apply()` method with lambda functions to apply custom text-cleaning routines (e.g., lowercasing, removing punctuation) to entire columns of text data.
- **Data Selection and Aggregation:** Mastered precise data selection using label-based indexing with `.loc[]` and integer-based indexing with `.iloc[]`. Applied the `.groupby()` method to aggregate data and compute statistics for different categories, a technique useful for analyzing text properties across different classes (e.g., comparing average sentence length in positive vs. negative reviews).

2.1.3 Matplotlib: The Framework for Data Visualization

Learned to use Matplotlib not just to create plots, but to answer specific questions about data and model behavior through visualization.

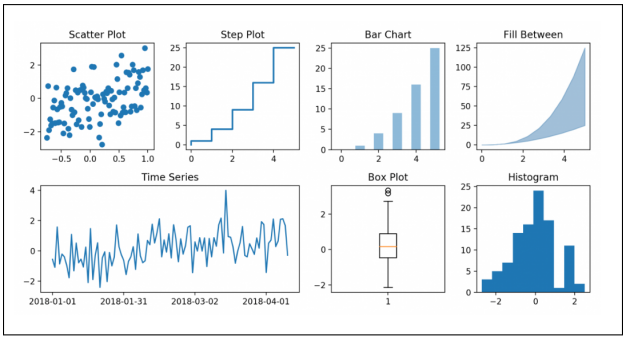


Figure 1: Plots made using Matplotlib

- **Exploratory Data Visualization:** Generated plots to gain initial insights. For instance, used `plt.hist()` to visualize the distribution of sentence lengths in a corpus, and `plt.bar()` to check for class imbalance in a classification dataset.
- **Model Performance Visualization:** A key practical skill was plotting the results of a model's training process. Used `plt.plot()` to create line graphs of training and validation loss/accuracy over epochs. This is the standard method for visually diagnosing overfitting and assessing model convergence.
- **Creating Publication-Quality Figures:** Beyond just generating a plot with `plt.show()`, learned to create complete and readable figures by adding critical context using `plt.title()`, `plt.xlabel()`, `plt.ylabel()`, and creating a `plt.legend()` to distinguish different data series.

2.2 Foundations in Machine Learning

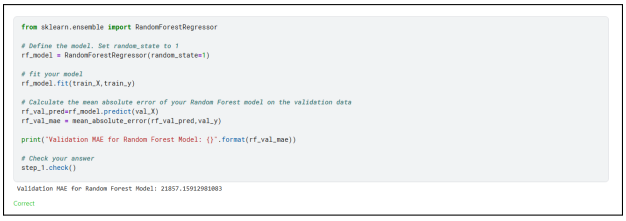
Intro to Machine Learning Before diving deep into the world of nlp a standard understanding of basics of machine learning model are needed which predict a particular attribute of a dataset by fitting datas into it.

2.2.1 The Scikit-learn API and Main Workflow

The primary skill obtained was proficiency in the standard scikit-learn library. This involves a consistent and a four-step pattern for using any of its machine learning algorithms which is known as "estimators".

I. Import the Model Class The first step is to import the specific algorithm to be used. For example, to use a Random Forest model, the syntax is: `from sklearn.ensemble import RandomForestRegressor`

II. Instantiate the Model After that instantiate the next Model , an instance of the model is created. This is where model parameter is specified also known as *hyper-parameters*. A common practice is to set a random state for reproducibility: `my model = RandomForestRegressor(random state=1)`



```
from sklearn.ensemble import RandomForestRegressor

# Define the model. Set random_state to 1
rf_model = RandomForestRegressor(random_state=1)

# fit your model
rf_model.fit(train_X,train_y)

# Calculate the mean absolute error of your Random Forest model on the validation data
rf_val_pred=rf_model.predict(val_X)
rf_val_mae = mean_absolute_error(rf_val_pred,val_y)

print("Validation MAE for Random Forest Model: {}".format(rf_val_mae))

# Check your answer
step_1.check()
```

```
Validation MAE for Random Forest Model: 21857.15912981083
```

Correct

Figure 2: Use of Random Forest Regressor

III. Fit the Model to Data This is the "training" step. The `.fit()` method is called on the model instance, passing the training data as arguments. This method is where the algorithm learns patterns from the data[6]. The syntax is consistently:
`my_model.fit(X_train, y_train)`
Here, `X_train` is the matrix of features (the input) and `y_train` is the vector of target values (the output we want to predict)[7].

IV. Predict on New Data Once the model is trained, the `.predict()` method is used to make predictions on new, unseen data (e.g., a validation or test set):
`predictions = my_model.predict(X_test)`

This simple, repeatable pattern forms the basis for applying a wide variety of machine learning models in practice.

2.2.2 A Practical Application: Predicting with a Regressor

These concepts were solidified by applying the full workflow to a regression problem, similar to predicting house prices from a set of features.

- **Data Preparation:** A key practical step learned was splitting the data into training and validation sets to properly evaluate the model. This is done using the `train_test_split` function:
  ```
  from sklearn.model_selection import train_test_split
  X_train, X_val, y_train, y_val = train_test_split(X, y)
  ```
- **Building and Training the Model:** A `RandomForestRegressor` was used as the chosen model. After instantiation, it was trained using the `.fit()` method exclusively on the training data (`X_train`, `y_train`)[2].
- **Making and Evaluating Predictions:** Predictions were made on the validation features (`X_val`). To judge the quality of these predictions, a performance metric was calculated. The Mean Absolute Error (MAE) was used as a straightforward way to measure the average error of the predictions:
  ```
  from sklearn.metrics import mean_absolute_error
  mae = mean_absolute_error(y_val, predictions)
  ```

```python
import pandas as pd
from sklearn.metrics import mean_absolute_error
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeRegressor


# Path of the file to read
iowa_file_path = '../input/home-data-for-ml-course/train.csv'

home_data = pd.read_csv(iowa_file_path)
# Create target object and call it y
y = home_data.SalePrice
# Create X
features = ['LotArea', 'YearBuilt', '1stFlrSF', '2ndFlrSF', 'FullBath', 'BedroomAbvGr', 'TotRmsAbvGrd']
X = home_data[features]

# Split into validation and training data
train_X, val_X, train_y, val_y = train_test_split(X, y, random_state=1)

# Specify Model
iowa_model = DecisionTreeRegressor(random_state=1)
# Fit Model
iowa_model.fit(train_X, train_y)

# Make validation predictions and calculate mean absolute error
val_predictions = iowa_model.predict(val_X)
val_mae = mean_absolute_error(val_predictions, val_y)
print("Validation MAE when not specifying max_leaf_nodes: {:,.0f}".format(val_mae))

# Using best value for max_leaf_nodes
iowa_model = DecisionTreeRegressor(max_leaf_nodes=100, random_state=1)
iowa_model.fit(train_X, train_y)
val_predictions = iowa_model.predict(val_X)
val_mae = mean_absolute_error(val_predictions, val_y)
print("Validation MAE for best value of max_leaf_nodes: {:,.0f}".format(val_mae))
```

Figure 3: A simple ML Model demonstrating above features

This exercise demonstrated the complete cycle: defining a problem, preparing data, training a model, and evaluating its performance on unseen data.

3 Neural Networks

Neural networks (NNs) are computational models which is inspired from biological neurons ane de signed to recognize patterns in data using weights and biases. They have become foundational in modern machine learning, powering applications in vision, language, and beyond than this.

3.1 Introduction to Neural Networks

Neural networks have a total of interconnected three layers:

- **Input Layer:**It takes raw data and initiates the network(e.g., pixel values, text embeddings)..
- **Hidden Layers:**Use activation functions anweighted connection to process data
- **Output Layer:** It generates predictions such as class labels or probabilities .

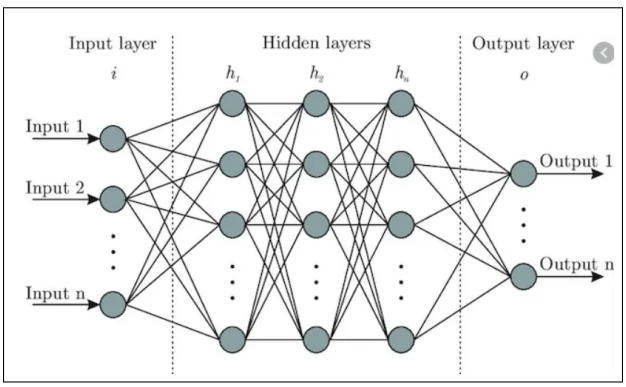


Figure 4: Visualization of Neural Networks

Neural networks learn by adjusting weights and biases via backpropagation methods using gradient descent to minimize the errors.Some kind of NN models like convolutional neural

networks (CNNs) excel at image classifications and recurrent neural networks (RNNs) or (long-short term memory)LSTM are suited for sequential data or seq2seq algorithm.

3.2 Activation Functions & Backpropagation

3.2.1 Activation Functions:

- **ReLU:** $\text{ReLU}(x) = \max(0, x)$, commonly used in hidden layers to avoid vanishing gradients.
- **Sigmoid:** Basically gives outputs tending to 1 for positive inputs and tending to 0 output while input is negative
- **Softmax:** Converts logits to probabilities for multi-class classification.

3.2.2 Backpropagation:

- Training proceeds via gradient descent.
- Steps: Compute loss (e.g., cross-entropy , SSR), calculate gradients using the chain rule, and update weights to minimize loss.

3.3 Attention Mechanisms

Attention enables models to focus on relevant parts of input data dynamically.

- **Query (Q):** Represents the current focus (e.g., a word being translated).
- **Key (K):** Encodes input elements (e.g., words in the source sentence).
- **Value (V):** Contains the actual content to be weighted.

Steps:

1. Scoring: Compute similarity between Q and K, e.g., $\text{score}(q, k) = q \cdot k^\top$.
2. Weighting: Apply softmax to get attention weights.
3. Aggregation: Compute weighted sum of V.

Types include self-attention (Q, K, V from the same sequence, as in transformers) and cross-attention (Q from one sequence, K/V from another, as in encoder-decoder models).

3.4 Transformers

Transformers, introduced in "Attention Is All You Need" (2017), replace recurrence with self-attention for efficient parallel processing.

3.4.1 Architecture:

- **Self-Attention Layer:** Computes attention weights for all input tokens simultaneously,

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V$$

- **Multi-Head Attention:** Multiple self-attention mechanisms run in parallel to capture diverse relationships.
- **Positional Encoding:** Injects sequence order information using sine/cosine functions.
- **Encoder-Decoder Stacks:** Encoders process input into context-rich representations; decoders generate output using encoder context and previous predictions.

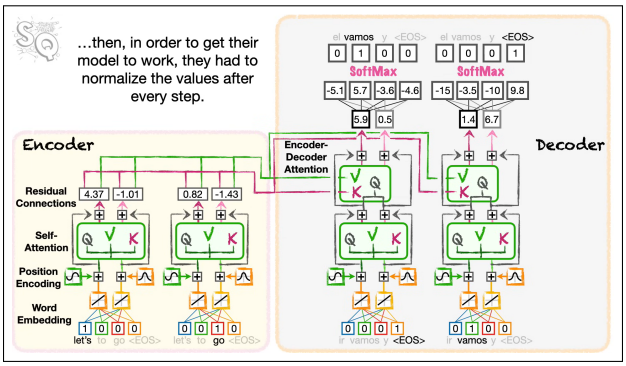


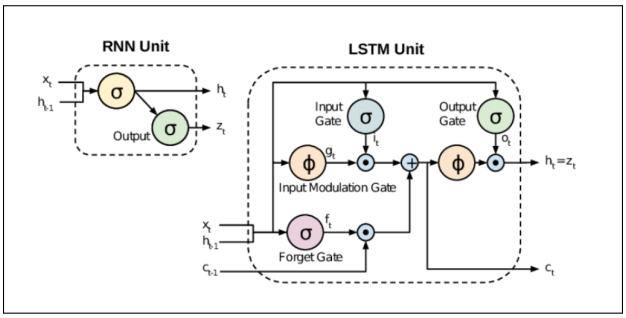
Figure 5: Super Simple structure of a Transformer

Transformers handle long-term dependencies better than RNNs and enable parallel computation, resulting in faster training and superior performance in many tasks.

3.5 Other Key Topics

3.5.1 RNNs & LSTMs:

- RNNs process sequences but struggle with long-term dependencies.
- LSTMs use gates (forget, input, output) to manage information flow, improving memory for longer sequences.



3.5.2 Word Embeddings(More details on NLP Section):

- Word2Vec generates dense word vectors using CBOW (context → target) or Skip-Gram (target → context) approaches.

3.5.3 Cross-Entropy Loss:

- Measures prediction error for classification:

$$\mathcal{L} = -\sum y_i \log(\hat{y}_i)$$

4 Word Embeddings and the Word2Vec Algorithm

4.1 Word Embeddings

Word embeddings map words to **dense vectors** in a continuous space R^d, where semantic and syntactic relationships are encoded through vector arithmetic. For example:

$$\mathbf{v}_{\text{tea}} = \begin{bmatrix} 0.24 \\ -1.76 \\ \vdots \\ 0.98 \end{bmatrix} \in R^{300}, \quad \mathbf{v}_{\text{coffee}} = \begin{bmatrix} 0.31 \\ -1.82 \\ \vdots \\ 1.02 \end{bmatrix} \in R^{300}$$

Here, $\mathbf{v}_{\text{tea}}$ and $\mathbf{v}_{\text{coffee}}$ are geometrically close due to their shared context (e.g., beverages).

4.2 Word2Vec: Mathematical Formulation

Word2Vec learns embeddings by leveraging the **distributional hypothesis**: words with similar contexts have similar meanings. The **skip-gram** variant predicts context words given a center word.

4.2.1 Probability Model:

For a center word c and context word o:

$$p(o|c) = \frac{\exp(\mathbf{u}_o^\top \mathbf{v}_c)}{\sum_{w \in V} \exp(\mathbf{u}_w^\top \mathbf{v}_c)}$$

- $\mathbf{v}_c \in R^d$: Center word vector (e.g., "tea").
- $\mathbf{u}_o \in R^d$: Context word vector (e.g., "drink").
- V: Vocabulary of all words.

4.2.2 Loss Function:

The **cross-entropy loss** for one (center, context) pair is:

$$\mathcal{L} = -\log p(o|c) = -\mathbf{u}_o^\top \mathbf{v}_c + \log \sum_{w \in V} \exp(\mathbf{u}_w^\top \mathbf{v}_c)$$

4.2.3 Gradient descent:

updates the vectors by computing partial derivatives:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{v}_c} = -\mathbf{u}_o + \sum_{w \in V} p(w|c)\mathbf{u}_w$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{u}_o} = -\mathbf{v}_c + \sum_{w \in V} p(w|c)\mathbf{v}_c$$

4.3 Negative Sampling

To avoid computing the full softmax denominator, **negative sampling** approximates the loss using k noise samples $\{n_1, ..., n_k\}$:

$$\mathcal{L} = -\log \sigma(\mathbf{u}_o^\top \mathbf{v}_c) - \sum_{i=1}^{k} \log \sigma(-\mathbf{u}_{n_i}^\top \mathbf{v}_c)$$

where $\sigma(x) = \frac{1}{1+e^{-x}}$. This converts the task into **binary classification**: distinguishing true context words from noise.

4.4 Geometric Properties

- **Cosine Similarity:** Measures semantic relatedness:

$$\text{sim}(\mathbf{v}_{\text{tea}}, \mathbf{v}_{\text{coffee}}) = \frac{\mathbf{v}_{\text{tea}}^\top \mathbf{v}_{\text{coffee}}}{\|\mathbf{v}_{\text{tea}}\| \|\mathbf{v}_{\text{coffee}}\|}$$

- **Linear Analogies:** Relationships emerge as vector offsets:

$$\mathbf{v}_{\text{king}} - \mathbf{v}_{\text{man}} + \mathbf{v}_{\text{woman}} \approx \mathbf{v}_{\text{queen}}$$

- **Orthogonality:** Unrelated words (e.g., $\mathbf{v}_{\text{tea}}$ and $\mathbf{v}_{\text{car}}$) have near-zero dot products.

4.5 Applications

- **Document Embeddings:** Aggregate word vectors (e.g., weighted average):

$$\mathbf{d}_{\text{document}} = \frac{1}{N} \sum_{i=1}^{N} \mathbf{v}_{w_i}$$

- **Classification:** Use a hyperplane $\mathbf{w}^\top \mathbf{v} + b = 0$ to separate categories (e.g., positive/negative sentiment).
- **Multilingual Alignment:** Learn linear transformations $\mathbf{W}$ to map embeddings across languages:

$$\mathbf{W}\mathbf{v}_{\text{tea}} \approx \mathbf{v}_{\text{thé}} \quad \text{(French)}$$

5 Improtant Libraries for Text Preprocessing

5.1 NLTK

NLTK (Natural Language Toolkit) is a foundational Python library for natural language processing . It provides a set of tools for linguistic data analysis including tokenization, stemming, lemmatization and part-of-speech (POS) tagging etc..

Key functionalities:

- **Tokenization:** Functions like `word_tokenize` and `sent_tokenize` are used to split text into words or sentences, forming the basis for further analysis.

- **Stemming:** Tools such as *PorterStemmer and SnowballStemmer* reduce words to their root forms which is useful for normalizing vocabulary in tasks like searching or clustering.
- **Lemmatization:** With `WordNetLemmatizer`, NLTK can convert words to their base dictionary form which ensures different inflections of a word are treated as equivalent.
- **POS Tagging:** The pos tag function assigns grammatical categories to each token, enabling syntactic and semantic analysis.

5.2 SpaCy

spaCy is a modern, high-performance NLP library which offers efficient, object-oriented pipelines with pre-trained models for multiple languages. **Key functionalities:**

- **Model Loading:**Models are loaded with spacy.load, providing instant access to tokenization, tagging, parsing, and NER capabilities.
- **Text Processing:** Passing text to the loaded model returns a Doc object, which contains tokens, lemmas, POS tags, dependency parses, and named entities, all accessible as attributes.
- **Tokenization:** Token objects can be iterated over directly from the Doc, with attributes like text, lemma , pos , and dep for linguistic analysis.

6 Text Preprocessing in NLP

Text preprocessing is the foundational step in any natural language processing (NLP) pipeline. Its goal is to clean and standardize raw text so that it can be effectively analyzed and modeled by machine learning algorithms. Proper preprocessing reduces noise, improves consistency and often boosts the performance of downstream NLP tasks such as classification, sentiment analysis and topic modeling.

6.1 Lower Casing

Converts all text to lowercase to standardize and reduce duplication (e.g., “Text” and “text” are treated the same).
Tip: Useful for most tasks, but sometimes case conveys meaning (e.g., Named Entity Recognition).

6.2 Removal of Punctuations

Removes punctuation marks to focus on the content words and standardize text.

6.3 Removal of Stopwords

Removes common words (like ,“the”, “is”, “and”) that usually carry little semantic value for tasks like classification.

```
import string
PUNCT_TO_REMOVE = string.punctuation
def remove_punctuation(text):
    return text.translate(str.maketrans('', '', PUNCT_TO_REMOVE))
df["text_wo_punct"] = df["text"].apply(remove_punctuation)
```

Figure 6: Example of punctuation removal in text preprocessing.

Note: Retain stopwords for tasks like POS(Basically tagging part of speech during lemmatization) tagging where they matter.

6.4 Removal of Frequent and Rare Words

Removes extremely frequent (domain-specific, e.g., "please", "DM") or rare words (appearing less than a set threshold) to reduce noise.
Tip: Especially useful for custom corpora or topic modeling.

6.5 Stemming

Reduces words to their root form by chopping off suffixes (e.g., "running" → "run").
Stemming is fast but may produce non-words (e.g., "private" → "privat").

```
from nltk.stem.porter import PorterStemmer
stemmer = PorterStemmer()
def stem_words(text):
    return " ".join([stemmer.stem(word) for word in text.split()])
df["text_stemmed"] = df["text"].apply(stem_words)
```

Figure 7: Illustration of Stemming applied to text.

6.6 Lemmatization

Reduces words to their lemma (base form), ensuring the result is a valid word.
Lemmatization is slower but more accurate than stemming. For best results, provide POS tags.

```
from nltk.stem import WordNetLemmatizer
lemmatizer = WordNetLemmatizer()
def lemmatize_words(text):
    return " ".join([lemmatizer.lemmatize(word) for word in text.split()])
df["text_lemmatized"] = df["text"].apply(lemmatize_words)
```

Figure 8: Illustration of lemmatization applied to text.

6.7 Removal of Emojis and Emoticons

Removes or converts emojis/emoticons depending on the task. For sentiment analysis, consider converting them to words instead of removing, as they often carry emotional meaning.

6.8 Spelling Correction

Corrects misspelled words to their proper form, improving consistency. Spelling correction can be slow on large datasets.

```
from spellchecker import SpellChecker
spell = SpellChecker()
def correct_spelling(text):
    return " ".join([spell.correction(word) for word in text.split()])
df["text_corrected"] = df["text"].apply(correct_spelling)
```

Figure 9: Example of spelling correction in text preprocessing.

7 Introduction to Tokenization

Tokenization is a foundational step in natural language processing (NLP) that involves splitting text into smaller manageable units called tokens. This process transforms raw, unstructured text into a structured format that is easier for machines to analyze which enables algorithms to identify patterns, parse meaning and perform tasks like classification and sentiment analysis etc.:

- **Word tokenization:** Splits text into words (e.g., "Chatbots are helpful." → ["Chatbots", "are", "helpful"]).
- **Character tokenization:** Splits text into characters (e.g., "Chat" → ["C", "h", "a", "t"]).
- **Subword tokenization:** Splits words into meaningful subunits (e.g., "Chatbots" → ["Chat", "bots"]).

7.1 Basic Tokenization Features Using NLP (with NLTK)

NLTK is a classic Python library for NLP that provides robust tokenization tools for both words and sentences. It can handle punctuation, contractions, and language-specific rules, making it ideal for experimentation and prototyping.

```
from nltk.tokenize import word_tokenize, sent_tokenize

text = "Tokenization is crucial for NLP. Let's split this text into sentences
and words!"
sentences = sent_tokenize(text)
words = word_tokenize(text)
print(sentences)  # ['Tokenization is crucial for NLP.', "Let's split this text
into sentences and words!"]
print(words)      # ['Tokenization', 'is', 'crucial', 'for', 'NLP', '.', 'Let',
"'s", 'split', ...]
```

Figure 10: Tokenization using NLTK

7.2 Tokenization Using TextBlob Library

TextBlob is a beginner-friendly NLP library built on top of NLTK and Pattern. It provides easy access to tokenized sentences and words via simple attributes, making basic tokenization straightforward and ideal for rapid prototyping.

```python
from textblob import TextBlob

blob = TextBlob("Natural Language Processing is fun. Tokenization is easy
with TextBlob!")
print(blob.sentences)  # [Sentence("Natural Language Processing is fun."),
Sentence("Tokenization is easy with TextBlob!")]
print(blob.words)      # ['Natural', 'Language', 'Processing', 'is', 'fun',
'Tokenization', 'is', 'easy', 'with', 'TextBlob']
```

Figure 11: Tokenization using TextBlob

7.3 Some spaCy Tokenization

spaCy is a modern, production-ready NLP library known for its speed and accuracy. Its tokenizer is integrated into a powerful processing pipeline and can handle language-specific rules, punctuation, and special cases.

```python
import spacy
nlp = spacy.load('en_core_web_sm')

doc = nlp("Tokenization with spaCy is fast and accurate!")
tokens = [token.text for token in doc]
print(tokens)  # ['Tokenization', 'with', 'spaCy', 'is', 'fast', 'and',
'accurate', '!']
```

Figure 12: Tokenization using SpaCy

7.4 Gensim Library Tokenization

Gensim is a library designed for topic modeling and vector space analysis of large text corpora. It provides efficient, memory-independent tokenization and preprocessing utilities, making it suitable for handling and modeling very large datasets.

```python
from gensim.utils import tokenize

text = "Gensim makes large-scale text processing easy."
tokens = list(tokenize(text))
print(tokens)  # ['Gensim', 'makes', 'large', 'scale', 'text',
'processing', 'easy']
```

Figure 13: Tokenization using Gensim

8 Bag of Words Implementation

The Bag of Words (BoW) model is a fundamental text representation technique in natural language processing that converts text documents into numerical vectors based on word frequency. This approach treats each document as an unordered collection of words, ignoring grammar and word order while focusing on word occurrence patterns.

8.1 Core Concept and Mathematical Foundation

The BoW model represents text documents as vectors in a high-dimensional space where each dimension corresponds to a unique word in the vocabulary. Given a corpus of documents $D = \{d_1, d_2, ..., d_n\}$ and a vocabulary $V = \{w_1, w_2, ..., w_m\}$, each document d_i

is represented as a vector $\vec{v_i} = [c_{i1}, c_{i2}, ..., c_{im}]$ that c_{ij} represents the frequency of word w_j in document d_i.

8.2 Implementation Steps

The implementation of a BoW model involves three main phases: preprocessing, vocabulary construction, and vector representation.

8.2.1 Text Preprocessing

Text preprocessing is crucial for effective BoW implementation. The process includes tokenization, lowercasing, and removing punctuation and stopwords.

```
import nltk
import re
import numpy as np
from collections import Counter

def preprocess_text(text):
    # Convert to lowercase
    text = text.lower()

    # Remove punctuation and special characters
    text = re.sub(r'[^\w\s]', '', text)

    # Remove extra whitespaces
    text = re.sub(r'\s+', ' ', text)

    # Tokenization
    tokens = nltk.word_tokenize(text)

    # Remove stopwords (optional)
    from nltk.corpus import stopwords
    stop_words = set(stopwords.words('english'))
    tokens = [token for token in tokens if token not in stop_words]

    return tokens

# Example usage
sample_text = "The quick brown fox jumps over the lazy dog."
processed_tokens = preprocess_text(sample_text)
print(f"Original: {sample_text}")
print(f"Processed: {processed_tokens}")
```

Listing 1: Comprehensive text preprocessing for BoW.

8.2.2 Vocabulary Construction and Frequency Analysis

After preprocessing, we construct the vocabulary by identifying all unique words and analyzing their frequencies.

```
def create_vocabulary(documents, max_features=1000):

    all_words = []
    processed_docs = []

```

```python
    # Process all documents
    for doc in documents:
        tokens = preprocess_text(doc)
        processed_docs.append(tokens)
        all_words.extend(tokens)

    # Count word frequencies
    word_freq = Counter(all_words)

    # Select top frequent words
    vocabulary = dict(word_freq.most_common(max_features))

    return vocabulary, processed_docs

# Example with sample documents
sample_documents = [
    "I love this movie. It's fantastic and amazing.",
    "This movie is terrible. I hate it completely.",
    "The acting was good but the plot was boring.",
    "Amazing cinematography and excellent storytelling."
]

vocab, processed_docs = create_vocabulary(sample_documents)
print("Vocabulary:", list(vocab.keys()))
```

Listing 2: Vocabulary construction and frequency analysis.

8.2.3 Vector Representation Construction

The final step involves converting documents into numerical vectors based on the vocabulary .

```python
def create_bow_vectors(processed_docs, vocabulary):
    vocab_list = list(vocabulary.keys())
    bow_vectors = []

    for doc_tokens in processed_docs:
        # Initialize vector with zeros
        vector = [0] * len(vocab_list)

        # Count occurrences of each vocabulary word
        for token in doc_tokens:
            if token in vocab_list:
                idx = vocab_list.index(token)
                vector[idx] += 1

        bow_vectors.append(vector)

    return np.array(bow_vectors), vocab_list

# Create BoW vectors
bow_matrix, feature_names = create_bow_vectors(processed_docs, vocab)
print("BoW Matrix shape:", bow_matrix.shape)
print("First document vector:", bow_matrix[0])
```

Listing 3: Creating BoW vectors from documents.

8.3 Advantages and Limitations

The BoW model offers simplicity and computational efficiency, making it suitable for baseline implementations. However, it suffers from several limitations, including the loss of semantic meaning, high dimensionality (curse of dimensionality), and an inability to capture word order or context.

9 TF-IDF Vectorization

Term Frequency-Inverse Document Frequency (TF-IDF) is an advanced text representation technique that addresses some limitations of the basic BoW model by weighting terms based on their importance within a document and across the corpus.

9.1 Mathematical Foundation

TF-IDF combines two components:

Term Frequency (TF): Measures how frequently a term appears in a document.

$$TF(t, d) = \frac{\text{Number of times term } t \text{ appears in document } d}{\text{Total number of terms in document } d}$$

Inverse Document Frequency (IDF): Measures the importance of a term across the corpus.

$$IDF(t) = \log\left(\frac{\text{Total number of documents}}{\text{Number of documents containing term } t + 1}\right)$$

(Note: a +1 is often added to the denominator to avoid division by zero).

TF-IDF Score:

$$\text{TF-IDF}(t, d) = TF(t, d) \times IDF(t)$$

9.2 Implementation with Scikit-learn

Scikit-learn provides an efficient implementation of TF-IDF through its ‘TfidfVectorizer‘ class.

```python
from sklearn.feature_extraction.text import TfidfVectorizer
import pandas as pd

def implement_tfidf(documents, max_features=1000):
    tfidf_vectorizer = TfidfVectorizer(
        max_features=max_features,
        stop_words='english',
        lowercase=True
    )

    tfidf_matrix = tfidf_vectorizer.fit_transform(documents)
    feature_names = tfidf_vectorizer.get_feature_names_out()

    return tfidf_matrix, feature_names

# Example implementation
```

```
documents = [
    "Machine learning is fascinating and powerful.",
    "Deep learning models require large datasets.",
    "Natural language processing uses machine learning.",
    "Text classification is an important NLP task."
]

tfidf_matrix, features = implement_tfidf(documents)

# Display results as a DataFrame
df_tfidf = pd.DataFrame(tfidf_matrix.toarray(), columns=features)
print("TF-IDF Matrix:")
print(df_tfidf)
```

Listing 4: Implementing TF-IDF with Scikit-learn.

10 Basic Sentiment Analysis

Sentiment analysis is a classification task that determines the emotional tone or opinion expressed in text [3]. Using the BoW and TF-IDF representations, we can build effective sentiment classifiers.

10.1 Data Preparation and Feature Engineering

```
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import MultinomialNB
from sklearn.metrics import accuracy_score, classification_report

def prepare_sentiment_data():
    texts = [
        "I love this product, it's amazing!",
        "This is the worst experience ever.",
        "Great quality and fast delivery.",
        "Terrible customer service, very disappointed.",
        "Excellent features and user-friendly interface.",
        "Poor quality, waste of money.",
    ]
    labels = [1, 0, 1, 0, 1, 0]  # 1: positive, 0: negative
    return texts, labels

def build_sentiment_classifier(texts, labels):
    """Build and evaluate a sentiment analysis classifier."""
    X_train, X_test, y_train, y_test = train_test_split(
        texts, labels, test_size=0.3, random_state=42, stratify=labels
    )

    vectorizer = TfidfVectorizer(stop_words='english')
    X_train_vec = vectorizer.fit_transform(X_train)
    X_test_vec = vectorizer.transform(X_test)

    classifier = MultinomialNB()
    classifier.fit(X_train_vec, y_train)

    y_pred = classifier.predict(X_test_vec)

```

```
    accuracy = accuracy_score(y_test, y_pred)
    print(f"Accuracy: {accuracy:.4f}")
    print("Classification Report:")
    print(classification_report(y_test, y_pred))

    return classifier, vectorizer

# Example usage
texts, labels = prepare_sentiment_data()
model, vectorizer = build_sentiment_classifier(texts, labels)
```

Listing 5: Building a sentiment analysis classifier.

10.2 Predicting Sentiment on New Text

```
def predict_sentiment(text, model, vectorizer):
    text_vec = vectorizer.transform([text])
    prediction = model.predict(text_vec)[0]
    sentiment = "Positive" if prediction == 1 else "Negative"
    return sentiment

# Example prediction
new_text = "The performance was outstanding and highly recommended!"
predicted_sentiment = predict_sentiment(new_text, model, vectorizer)
print(f'Text: "{new_text}"')
print(f'Predicted Sentiment: {predicted_sentiment}')
```

Listing 6: Predicting sentiment for new text.

11 Transformer Architecture

The Transformer architecture, introduced in "Attention Is All You Need", revolutionized NLP by relying entirely on attention mechanisms, forgoing recurrent or convolutional layers.

11.1 Core Components

11.1.1 Multi-Head Attention Mechanism

Multi-head attention allows the model to jointly attend to information from different representation subspaces at different positions. The core is the scaled dot-product attention:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Where Q (queries), K (keys), and V (values) are linear projections of the input.

```
import torch
import torch.nn as nn
import math

class MultiHeadAttention(nn.Module):
    def __init__(self, d_model, num_heads):
        super(MultiHeadAttention, self).__init__()
```

```
        assert d_model % num_heads == 0
        self.d_model = d_model
        self.num_heads = num_heads
        self.d_k = d_model // num_heads

        self.W_q = nn.Linear(d_model, d_model)
        self.W_k = nn.Linear(d_model, d_model)
        self.W_v = nn.Linear(d_model, d_model)
        self.W_o = nn.Linear(d_model, d_model)

    def forward(self, query, key, value, mask=None):
        batch_size = query.size(0)

        Q = self.W_q(query).view(batch_size, -1, self.num_heads, self.d_k).transpose(1, 2)
        K = self.W_k(key).view(batch_size, -1, self.num_heads, self.d_k).transpose(1, 2)
        V = self.W_v(value).view(batch_size, -1, self.num_heads, self.d_k).transpose(1, 2)

        scores = torch.matmul(Q, K.transpose(-2, -1)) / math.sqrt(self.d_k)
        if mask is not None:
            scores = scores.masked_fill(mask == 0, -1e9)

        attention_weights = torch.nn.functional.softmax(scores, dim=-1)
        attention_output = torch.matmul(attention_weights, V)

        attention_output = attention_output.transpose(1, 2).contiguous().view(batch_size, -1, self.d_model)

        return self.W_o(attention_output)
```

Listing 7: PyTorch implementation of Multi-Head Attention.

11.1.2 Positional Encoding

Since Transformers do not use recurrence, positional encodings are added to the input embeddings to give the model information about the relative or absolute position of the tokens.

$$PE_{(pos,2i)} = \sin(pos/10000^{2i/d_{\text{model}}})$$

$$PE_{(pos,2i+1)} = \cos(pos/10000^{2i/d_{\text{model}}})$$

11.1.3 Complete Transformer Block

A full Transformer block consists of a multi-head attention layer followed by a feed-forward network, with residual connections and layer normalization after each.

```
class TransformerBlock(nn.Module):
    def __init__(self, d_model, num_heads, d_ff, dropout=0.1):
        super(TransformerBlock, self).__init__()
        self.attention = MultiHeadAttention(d_model, num_heads)
        self.norm1 = nn.LayerNorm(d_model)
        self.norm2 = nn.LayerNorm(d_model)
        self.feed_forward = nn.Sequential(
            nn.Linear(d_model, d_ff),
```

```
            nn.ReLU(),
            nn.Linear(d_ff, d_model)
        )
        self.dropout = nn.Dropout(dropout)

    def forward(self, x, mask=None):
        attn_output = self.attention(x, x, x, mask)
        x = self.norm1(x + self.dropout(attn_output))
        ff_output = self.feed_forward(x)
        x = self.norm2(x + self.dropout(ff_output))
        return x
```

Listing 8: A complete Transformer block.

12 Fine-tuning BERT

BERT (Bidirectional Encoder Representations from Transformers) is a pre-trained model that can be fine-tuned for various downstream tasks. Fine-tuning adapts the pre-trained weights to a specific task using a small amount of labeled data.

12.1 Fine-tuning Implementation with Hugging Face

The Hugging Face ‘transformers‘ library simplifies the process of fine-tuning BERT.

```
from transformers import BertTokenizer, BertForSequenceClassification,
    Trainer, TrainingArguments
from torch.utils.data import Dataset
import torch

class SentimentDataset(Dataset):
    def __init__(self, texts, labels, tokenizer, max_len):
        self.texts = texts
        self.labels = labels
        self.tokenizer = tokenizer
        self.max_len = max_len

    def __len__(self):
        return len(self.texts)

    def __getitem__(self, idx):
        text = str(self.texts[idx])
        label = self.labels[idx]
        encoding = self.tokenizer.encode_plus(
            text,
            add_special_tokens=True,
            max_length=self.max_len,
            return_token_type_ids=False,
            padding='max_length',
            truncation=True,
            return_attention_mask=True,
            return_tensors='pt',
        )
        return {
            'input_ids': encoding['input_ids'].flatten(),
            'attention_mask': encoding['attention_mask'].flatten(),
            'labels': torch.tensor(label, dtype=torch.long)
```

```
        }

# --- This is a conceptual example; running it requires a proper dataset ---
# texts, labels = prepare_sentiment_data()
# train_texts, val_texts, train_labels, val_labels = train_test_split(texts, labels, test_size=0.2)
#
# tokenizer = BertTokenizer.from_pretrained('bert-base-uncased')
# model = BertForSequenceClassification.from_pretrained('bert-base-uncased', num_labels=2)
#
# train_dataset = SentimentDataset(train_texts, train_labels, tokenizer, max_len=128)
# val_dataset = SentimentDataset(val_texts, val_labels, tokenizer, max_len=128)
#
# training_args = TrainingArguments(
#     output_dir='./results',
#     num_train_epochs=1,
#     per_device_train_batch_size=8,
#     per_device_eval_batch_size=8,
#     warmup_steps=500,
#     weight_decay=0.01,
#     logging_dir='./logs',
# )
#
# trainer = Trainer(
#     model=model,
#     args=training_args,
#     train_dataset=train_dataset,
#     eval_dataset=val_dataset
# )
#
# trainer.train()
```

Listing 9: Fine-tuning BERT for sequence classification.

13 End-to-End NLP Pipeline

An end-to-end NLP pipeline integrates all components from data preprocessing to model deployment, creating a complete system for real-world applications.

13.1 Pipeline Architecture

A typical pipeline consists of:

1. **Data Acquisition**: Collecting raw text data.
2. **Text Preprocessing**: Cleaning and standardizing text.
3. **Feature Engineering**: Using techniques like TF-IDF or embeddings.
4. **Model Training**: Training a classifier on the features.
5. **Model Evaluation**: Assessing performance on a test set.

6. **Model Deployment**: Making the model available for predictions.

13.2 Complete Pipeline Implementation

We can use ‘scikit-learn‘’s ‘Pipeline‘ object to chain these steps together.

```
import joblib
from sklearn.pipeline import Pipeline

# (Assumes texts and labels are loaded)
# texts, labels = prepare_sentiment_data()
# X_train, X_test, y_train, y_test = train_test_split(texts, labels,
    test_size=0.2, random_state=42)

# Define the pipeline
nlp_pipeline = Pipeline([
    ('vectorizer', TfidfVectorizer(stop_words='english', ngram_range
    =(1, 2))),
    ('classifier', MultinomialNB())
])

# Train the pipeline
print("Training the pipeline...")
nlp_pipeline.fit(X_train, y_train)
print("Training complete.")

# Evaluate the pipeline
accuracy = nlp_pipeline.score(X_test, y_test)
print(f"Pipeline Accuracy: {accuracy:.4f}")

# Make a prediction
new_text = "This movie was incredibly well-made and inspiring."
prediction = nlp_pipeline.predict([new_text])[0]
sentiment = "Positive" if prediction == 1 else "Negative"
print(f'Prediction for "{new_text}": {sentiment}')

# Save the trained pipeline
joblib.dump(nlp_pipeline, 'sentiment_pipeline.pkl')
print("Pipeline saved to sentiment_pipeline.pkl")

# Load and use the pipeline
loaded_pipeline = joblib.load('sentiment_pipeline.pkl')
loaded_prediction = loaded_pipeline.predict([new_text])[0]
print(f"Loaded pipeline prediction is correct: {loaded_prediction ==
    prediction}")
```

Listing 10: An end-to-end sentiment analysis pipeline.

14 Model Deployment Basics

Model deployment is the process of integrating a trained model into a production environment. A common method is to expose the model via a REST API.

14.1 Deployment with FastAPI

FastAPI is a modern, fast web framework for building APIs with Python.

```python
# Save this code as api.py
from fastapi import FastAPI
from pydantic import BaseModel
import joblib

# Define the input data model
class TextIn(BaseModel):
    text: str

# Define the output data model
class PredictionOut(BaseModel):
    sentiment: str

app = FastAPI()

# Load the trained pipeline
pipeline = joblib.load('sentiment_pipeline.pkl')

@app.post("/predict", response_model=PredictionOut)
def predict(payload: TextIn):
    prediction = pipeline.predict([payload.text])[0]
    sentiment = "Positive" if prediction == 1 else "Negative"
    return {"sentiment": sentiment}

# To run this API:
# 1. Install FastAPI and Uvicorn: pip install fastapi "uvicorn[standard
    ]"
# 2. Run the server: uvicorn api:app --reload
```

Listing 11: Deploying the NLP pipeline with FastAPI.

14.2 Dockerization for Portability

Docker allows you to package the application and its dependencies into a container, ensuring it runs consistently across different environments.

Dockerfile

```dockerfile
# Use an official Python runtime as a parent image
FROM python:3.9-slim

# Set the working directory in the container
WORKDIR /app

# Copy the requirements file into the container
COPY requirements.txt .

# Install any needed packages specified in requirements.txt
RUN pip install --no-cache-dir -r requirements.txt

# Copy the rest of the application code
COPY . .

# Make port 8000 available to the world outside this container
EXPOSE 8000

# Run api.py when the container launches
```

```
CMD ["uvicorn", "api:app", "--host", "0.0.0.0", "--port", "8000"]
```

Listing 12: Dockerfile for the FastAPI application.

requirements.txt

```
fastapi
uvicorn[standard]
scikit-learn
joblib
```

Build and Run Commands

```
# Build the Docker image
docker build -t nlp-api .

# Run the Docker container
docker run -d -p 8000:8000 --name sentiment-service nlp-api
```